



TravelGuard AI

From concept to a working decision engine.

A running prototype that takes a live transaction, queries CAMARA network signals, scores the risk with an AI agent, and returns an explainable approve / step-up / reject decision in under a second.

01 Mock bank sends transaction



02 CAMARA signals collected



03 Explainable decision returned

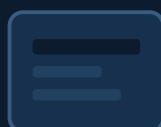
Working prototype

CAMARA sandbox

FastAPI + React

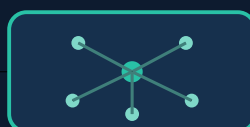
AI risk engine

3 demo scenarios



Mock bank API

Transaction request



TravelGuard engine

CAMARA + AI scoring



Decision + reason

Returned to the bank

1 What We Built

The scope of the prototype, and what it proves.

The Idea Phase argued that the mobile network already knows whether a cardholder is genuinely abroad. The prototype turns that argument into **running code**: a decision service a bank can call, backed by CAMARA signals and an AI risk engine.

4

CAMARA APIs orchestrated per decision

3

End-to-end demo scenarios: approve, step-up, reject

1

REST endpoint the bank integrates against

In scope for the prototype

Built and working

- Decision API (POST /v1/decision)
- CAMARA signal collector with fallback handling
- AI risk engine: feature builder, scoring, fusion rules
- Explanation generator producing audit text
- Bank-side demo dashboard showing live decisions
- Customer profile store for behavioural baselines

Deliberately out of scope

- Production card-network integration (ISO 8583 / VisaNet)
- Real customer PII — all profiles are synthetic
- Full regulatory certification and pen-testing
- Multi-operator commercial aggregation
- Model training on real historical fraud data

Real versus simulated

Being explicit about this matters, because it is the first question a bank or an operator will ask.

Component	Status in Prototype	Path to Production
CAMARA API calls	Sandbox / Network-as-Code	Swap base URL and credentials to the MNO's live gateway
Bank fraud engine	Mock service we wrote	Replaced by the bank's real engine calling our endpoint
Cardholder profiles	Synthetic dataset	Bank supplies its own behavioural features
Risk scoring logic	Implemented and running	Retrained on the bank's labelled fraud history
Explanation output	Implemented and running	Mapped to the bank's audit and dispute format

2 Why Would a Bank Add This?

The real question behind this hackathon, answered directly.

Every issuing bank already runs a Fraud Engine. It scores country, amount, merchant, history, velocity and device — and it does that well. So why would BNP Paribas plug in another layer at all?

What the Fraud Engine cannot see

The Fraud Engine is excellent at everything **inside** the bank's own data: card history, spending pattern, merchant risk, velocity. But it has one permanent blind spot — it has no reliable way to know **where the cardholder physically is, right now**. Today it can only guess, from sources that are weak by construction:

WHAT THE BANK TRIES TODAY	WHY IT IS NOT EVIDENCE
IP address of the merchant / app session	Defeated by a VPN or a merchant's own hosting location
GPS from the banking app	Requires the app to be open, and is trivially spoofable
Travel notice set by the customer	Forgotten, incomplete, or ignored by the engine anyway — this is exactly what happened to Ahmed

TravelGuard does not compete with the Fraud Engine. It hands it a signal the bank structurally cannot produce on its own: a live, network-level confirmation of the customer's location, attested by the operator — not the app, not the IP, not a form the customer filled in before leaving.

The same numbers, read differently

This is not a new decision path bolted on beside the old one. It is a new input feeding the same fraud logic the bank already trusts:

SIGNAL THE FRAUD ENGINE SEES	WITHOUT TRAVELGUARD	WITH TRAVELGUARD
New country + high amount	Risk 90 → Reject	Network confirms genuine roaming → Risk 12 → Approve
New country + recent SIM change	Risk 40 → borderline, often approved	Takeover pattern surfaced → Risk 94 → Reject

Three reasons this signal is worth adding, specifically:

Hard to spoof

Confirmed by the operator's network, not by the customer's device or app

Not in the bank's reach

No issuing bank has direct access to telecom network state

Changes the meaning of existing signals

The same rule engine, with one blind spot removed

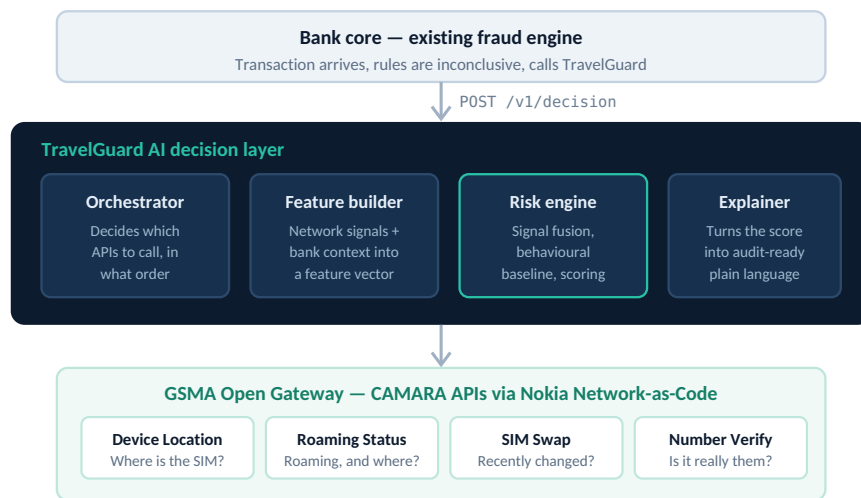
What we are not claiming

We are not claiming TravelGuard replaces fraud expertise the bank has built over years, and we are not asking the bank to hand over its decision. The bank keeps the Fraud Engine, keeps the final call, and keeps the compliance ownership. TravelGuard's job is narrower and more honest: answer the one question — *is this customer genuinely where the transaction says they are* — that no amount of banking data alone can answer.

3 System Architecture

Four layers, one synchronous call.

The bank never talks to CAMARA directly. It makes a single call to TravelGuard, and we handle orchestration, degradation, scoring and explanation behind that boundary.



Design principle: the network layer is replaceable. Every CAMARA call sits behind an adapter interface, so moving from sandbox to a live operator gateway is a configuration change, not a rewrite.

Latency budget

A payment authorisation will not wait, so every stage is bounded. If a stage exceeds its allowance the engine proceeds with what it has rather than blocking the transaction on its own slowness.

STAGE	ALLOWANCE	BEHAVIOUR ON TIMEOUT
Request validation and profile lookup	~50 ms	Fail closed — malformed requests are rejected outright
CAMARA signal collection (parallel)	~400 ms	Proceed with whichever signals returned; mark the rest unavailable
Feature build and scoring	~80 ms	Fall back to the conservative rule overlay
Explanation generation	~60 ms	Emit the structured reason codes without the narrative text

4 CAMARA Integration

Which APIs we call, why, and what we do when they fail.

CAMARA API	WHAT WE ASK IT	HOW THE ANSWER IS USED
Device Location Verification	Is this device within the country where the transaction is being made?	Primary evidence of genuine travel. A match is the single strongest approve signal we have.
Roaming Status	Is the subscriber roaming, and in which country?	Corroborates location and adds duration context — roaming since yesterday reads differently from roaming since an hour ago.
SIM Swap	When was the SIM last changed for this number?	Inverts the meaning of everything else. A recent swap turns a reassuring location match into a takeover indicator.
Number Verification	Does this number belong to the device making the request?	Binds the network signals to the cardholder's registered identity, silently and without an OTP.

Cost-aware orchestration

CAMARA calls are billable, so the orchestrator does not fire all four every time. It escalates only as far as the uncertainty requires:

- 1 **Always** — Roaming Status. Cheapest signal, resolves the majority of clear-cut travel cases.
- 2 **If roaming country matches the merchant country** — SIM Swap, to rule out takeover before approving.
- 3 **If the signals disagree or the amount is high** — Device Location, for precise confirmation.
- 4 **If identity itself is in question** — Number Verification, before any step-up is offered.

Degradation strategy

A network API can time out, return partial data, or be unavailable for a given operator. The prototype treats missing data as **uncertainty, not as guilt**: absent signals widen the confidence interval and push borderline cases toward step-up authentication rather than toward rejection. Every call has a hard timeout, and the engine always returns a decision within its latency budget even if one or more signals never arrive.

Why this matters commercially: selective calling is what makes the economics work for a bank. Paying for four API calls on every low-value coffee purchase would not survive a procurement review; paying for one, and escalating only when it is genuinely ambiguous, will.

5 The AI Risk Engine

How signals become a score, and a score becomes a decision.

Feature vector

Network signals and bank context are combined into one vector before any scoring happens. Neither source is sufficient alone.

From the network

- Country match between device and merchant
- Roaming state and time since roaming began
- Days since last SIM change
- Number-to-device binding confirmed
- Signal completeness score

From the bank

- Countries visited in the last 24 months
- Amount relative to the customer's own distribution
- Merchant category familiarity
- Time since last domestic transaction
- Historical dispute and chargeback flags

Fusion rules that a flat rule engine cannot express

These are interaction effects: the meaning of one signal is conditional on another.

SIGNAL COMBINATION	NAIVE READING	TRAVELGUARD READING
Roaming in country + SIM swapped 1 day ago	Two signals, one good one neutral	Takeover pattern — escalate sharply
Roaming in country + SIM stable 8 months + frequent traveller	Foreign country, therefore suspicious	Entirely normal for this customer — approve
Location unavailable + roaming confirmed + small amount	Incomplete data, therefore block	Enough evidence for a low-value approval
Device at home + transaction abroad	Possible travel notice	Card is being used where the customer is not

Thresholds and actions

0 – 30	Approve	Sufficient converging evidence. The customer experiences nothing at all.
31 – 70	Step-up	Plausible but unconfirmed. Ask the customer, then decide with their answer as an input.
71 – 100	Reject	Evidence actively contradicts the transaction. Block and flag for investigation.

Thresholds are configurable per bank, because risk appetite is a commercial decision rather than a technical one. A private bank and a prepaid travel card issuer will not sit at the same cut-off.

6 Integration Contract

One endpoint. The bank's entire integration surface.

Integration effort is the main adoption barrier for a bank, so the contract is deliberately minimal: a single synchronous POST, returning a decision the fraud engine can act on directly.

Request

```
POST /v1/decision
Authorization: Bearer <bank_api_key>

{
  "transaction_id": "txn_8f21c",
  "msisdn":       "+4915112345678",
  "amount":       { "value": 4.20, "currency": "EUR" },
  "merchant":     { "country": "MA", "category": "restaurant" },
  "customer_ref": "cust_00417"
}
```

Response

```
{
  "decision": "APPROVE",
  "risk_score": 18,
  "confidence": "high",
  "signals_used": ["roaming_status", "sim_swap"],
  "explanation": [
    "Active roaming detected in MA at time of transaction",
    "SIM unchanged for 247 days",
    "Customer has transacted in MA in 3 previous trips",
    "Amount is within this customer's normal range"
  ],
  "latency_ms": 412
}
```

Design decisions worth noting

The bank sends no raw PII

We receive an MSISDN and an opaque customer reference. Behavioural features stay on the bank's side; we never hold names, card numbers or addresses.

The response is advisory

The bank retains the final decision. We return a recommendation and its justification — which is also what keeps the regulatory responsibility where it already sits.

Explanation is a first-class field, not a log line. It is what lets a call-centre agent tell a customer why their payment was stopped, and what a regulator will ask to see. Any model we deploy has to be able to produce it.

7 Demo Scenarios

Three transactions, three different answers, all explainable.

APPROVE**Ahmed buys coffee in Marrakech**

Risk 18

EUR 4.20 · restaurant · MA · German card

- Roaming Status: roaming in MA, active for 6 hours
- SIM Swap: last change 247 days ago
- Profile: has travelled to MA on three previous occasions
- Amount: within his normal daily spend

Two API calls were enough. Device Location was never queried, because the evidence already converged. **The transaction the old rule engine would have blocked goes through untouched.**

STEP-UP**First-time traveller, high-value purchase**

Risk 52

EUR 1,850 · electronics · TR · no prior travel history

- Roaming Status: roaming in TR, active for 40 minutes
- Device Location: confirmed in TR, escalated because of the amount
- SIM Swap: last change 12 days ago — recent, but not immediate
- Profile: no international transactions on record

The network says the customer is there. The behavioural baseline says nothing like this has ever happened. Rather than guess, the engine asks: a push confirmation is sent, and the answer feeds back into the decision.

REJECT**SIM-swap takeover attempt**

Risk 94

EUR 2,400 · money transfer · MA · card reported active at home

- SIM Swap: SIM changed 14 hours ago
- Device Location: device is in the home country, not in MA
- Number Verification: binding could not be confirmed
- Merchant category: high-risk transfer, not typical for this customer

Every individual signal here would look survivable to a threshold rule. Read together, they describe a textbook account takeover — and this is precisely the case a location-only check would have waved through.

8 Validation & Limitations

What the prototype demonstrates, and what it does not yet prove.

How we tested it

We built a synthetic transaction set covering the cases that matter: genuine travellers of varying travel frequency, takeover attempts, degraded-signal conditions, and ordinary domestic activity that should never be touched. Each case was run end to end through the live endpoint.

WHAT WE MEASURED	WHY IT IS THE RIGHT MEASURE	RESULT
False-positive rate on genuine travellers	This is the entire problem statement — the metric the bank feels in its call centre	[your figure]
Detection rate on takeover cases	Reducing blocks is worthless if fraud starts getting through	[your figure]
End-to-end decision latency	A payment authorisation cannot wait; the budget is unforgiving	[your figure]
CAMARA calls per decision	Determines whether the unit economics work at scale	[your figure]
Decisions with a complete explanation	An unexplainable decision is not deployable in a regulated bank	[your figure]

Replace the bracketed cells with the figures from your own test run before submitting — judges will trust a modest measured number far more than an impressive unmeasured one.

Honest limitations

What we cannot claim yet

- No validation against real labelled fraud data
- Synthetic profiles cannot capture true behavioural variance
- Sandbox latency is not live-network latency
- Single-operator testing only

Open dependencies

- CAMARA coverage varies by operator and market
- Consent and data-protection framework per jurisdiction
- Commercial pricing per API call is not yet fixed
- Inbound roamers depend on the visited network

The coverage question is the real one. TravelGuard only helps a customer whose operator exposes CAMARA APIs. That makes operator adoption across MENA a prerequisite for the product — and equally, a concrete commercial reason for operators to adopt.

9 From Prototype to Pilot

What happens after the hackathon.

- 1 **Shadow mode with one bank** — we score transactions alongside the existing engine without affecting outcomes, and compare decisions
- 2 **Retrain on real labelled data** — replace synthetic baselines with the bank's own fraud history
- 3 **Live pilot on a travel-heavy segment** — inbound tourists or a corporate travel card portfolio
- 4 **Second operator, second market** — prove the adapter layer holds across gateways
- 5 **Commercial rollout** — per-decision pricing shared between issuer and operator

Technology stack

Service

Python, FastAPI, async CAMARA adapters with per-call timeouts

Intelligence

Feature pipeline, gradient-boosted scoring, rule-based fusion overlay, LLM explanation layer

Demo surface

React dashboard showing live decisions, signals and reasoning per transaction

What we would ask of an operator partner

- Production CAMARA access in one MENA market
- Realistic latency and availability figures
- Indicative per-call commercial terms
- An introduction to one issuing bank for shadow mode

WHERE THIS GOES

The prototype proves one transaction can be decided correctly. The product is the layer that decides every cross-border transaction in the region correctly — and turns network truth into a service banks pay for.